

Streaming and Incremental Pipelines

Scenario



ShopNow e-commerce store

In previous labs, we tried to support ShopNow assuming an old-school "Data at Rest" paradigm. We processed batches of files, optimized storage on MinIO, and orchestrated tasks using Flyte to ensure idempotency. However, in a real-world scenario like ShopNow, business requirements often evolve beyond "yesterday's data":

- Marketing wants to see the impact of a flash sale every 15 minutes
- Fraud Detection needs to identify suspicious patterns in seconds
- Cloud Infrastructure costs are rising because we are re-processing millions of old rows just to add a few thousand new ones (the full refresh anti-pattern).

In this tutorial, you will evolve the ShopNow architecture. You will move away from static CSV uploads and implement a hybrid pipeline where streaming, incremental, and batch processing coexist across the Medallion layers.

By the end of this tutorial, you will be able to:

- Produce and consume events: interface with an event-driven transport layer such as Kafka
- Implement streaming ingestion: map a live stream into a Bronze table for immediate "query-on-read" access
- Master incremental logic: use Iceberg snapshots and the MERGE INTO pattern to update Silver tables without full re-computes
- Manage time semantics: understand the difference between event time and processing time when dealing with late-arriving data

To solve ShopNow's problems, we will implement the following flow:

Layer	Input	Logic Type	Technology	Latency
Source	Python Simulator	Streaming	Kafka	Milliseconds
Bronze	Kafka Topic	Streaming Ingestion	Trino Kafka Connector	Seconds
Silver	Bronze Table	Incremental (Merge)	Flyte + Trino + Iceberg	~10 Minutes
Gold	Silver Table	Batch (Aggregate)	Flyte + Trino	Daily

Imagine the ShopNow store is live! Customers are browsing, adding items to carts, and checking out right now. Instead of waiting for a dump of logs at midnight, we want to see the Bronze layer filling up in real-time. We will then promote these events to Silver incrementally, ensuring that our clean, deduplicated data is never more than a few minutes old.

Environment setup and the event producer

!!! You can skip this section if you switch to version 1.3 of our stack !!!

To handle real-time events, we need a message broker. We will use Redpanda, a modern, high-performance, Kafka-compatible platform that is extremely easy to run in Docker. Add the following service to your existing `docker-compose.yml` file. This will spin up a single-node Redpanda cluster.

```
redpanda:
  image: docker.redpanda.com/redpandadata/redpanda:v23.2.1
  container_name: redpanda
  command:
    - redpanda start
    - --smp 1
    - --overprovisioned
    - --node-id 0
    - --kafka-addr internal://0.0.0.0:9092,external://0.0.0.0:19092
    - --advertise-kafka-addr
  internal://redpanda:9092,external://localhost:19092
  ports:
    - "19092:19092" # Kafka API
    - "18081:18081" # Schema Registry
    - "18082:18082" # HTTP Proxy
```

Once Redpanda is running, we need a "bucket" for our events. In Kafka terms, this is called a Topic. Open your terminal and run:

```
docker exec -it redpanda rpk topic create web_events
```

We need a library to talk to Kafka from Python. We will use `kafka-python-ng` (a modern fork) for its simplicity. Install it with `pip install kafka-python-ng`.

Now, let's look at the script that simulates user behavior (`producer.py`). This script will generate three types of events: `view_item`, `add_to_cart`, and `purchase`. To test our later "Incremental" logic, we will intentionally introduce duplicates and slight delays.

Run the producer in one terminal window:

```
python producer.py
```

Now, check if Redpanda is actually receiving them without writing a consumer yet. Use the `rpk` tool included in the container:

```
docker exec -it redpanda rpk topic consume web_events
```

You should see a scroll of JSON objects flying through your terminal. These are the Unbounded Data events that we will now need to capture into our Lakehouse.

Streaming ingestion

In this section, we will bridge the gap between the message broker (Redpanda/Kafka) and our analytical storage (Iceberg). We will use Trino as the ingestion engine to "land" the live events into our Bronze layer. We will start by mapping Kafka events to Trino.

The Trino Kafka connector needs to know how to interpret the raw bytes flowing through the `web_events` topic. We define this using a table description file (`web_events.json`). To enable the mapping, place the `web_events.json` file in your local `kafka_config/` directory and ensure your `docker-compose.yml` mounts this directory to `/etc/trino/kafka/` inside the Trino container. This allows the Trino engine to parse the raw JSON events into a structured table format.

!!! If you use version 1.3 of the Stack, this is already done !!!

We must now create our bronze table, which is the "landing zone". It must be an append-only record of everything that happened, including duplicates or malformed data. We also add an `ingestion_timestamp` to track when the data reached our platform. This is critical for incremental processing later.

Run the following SQL in trino:

```
CREATE TABLE iceberg.bronze.web_events_raw (
  event_id VARCHAR,
  event_type VARCHAR,
  customer_id VARCHAR,
  item_id VARCHAR,
  amount DOUBLE,
  event_time_raw VARCHAR,
  ingestion_timestamp TIMESTAMP(6) WITH TIME ZONE
)
WITH (
  format = 'PARQUET',
  partitioning = ARRAY['day(ingestion_timestamp)']
);
```

Now, let's pull the data from the live stream into the Iceberg table. In a real-time system, this would be a continuous process. Here, we execute a manual capture to see the transition. Run the following script:

```
INSERT INTO iceberg.bronze.web_events_raw
SELECT
  event_id,
```

```
event_type,
customer_id,
item_id,
amount,
event_time as event_time_raw,
now() as ingestion_timestamp
FROM kafka.default.web_events;
```

After running the insert, verify that the events have moved from the "volatile" Kafka topic to the "durable" MinIO storage:

```
SELECT * FROM iceberg.bronze.web_events_raw
ORDER BY ingestion_timestamp
DESC LIMIT 10;
```

TODO / Homework: To simulate a more realistic setting, write a script that regularly (e.g. every minute) pushes events from Kafka to Bronze. For instance, this could be a Flyte workflow with a launch plan, or a cron job that regularly runs the above SQL script. This should use the offset / timestamp of the last event to be retrieved, to avoid loading duplicate data into bronze (which is what happens now if you run the query multiple times). An example of solving this using a watermark is provided in the workflow used next.

Incremental Processing (Bronze to Silver)

In the Silver layer, our goal is to produce a clean, deduplicated, and normalized version of the truth. We will use a "Watermark" strategy to ensure we only process new events.

Unlike the Bronze layer, the Silver table enforces a schema and ensures that every event_id is unique. In Trino, run the following command:

```
CREATE TABLE iceberg.silver.web_events_clean (
  event_id VARCHAR,
  event_type VARCHAR,
  customer_id VARCHAR,
  item_id VARCHAR,
  amount DOUBLE,
  event_time TIMESTAMP(6) WITH TIME ZONE,
  ingested_at TIMESTAMP(6) WITH TIME ZONE
)
WITH (
  format = 'PARQUET',
  partitioning = ARRAY['day(event_time)']
);
```

To handle duplicates (which we intentionally created in the Producer) and late-arriving data, we use the MERGE statement. To avoid reading the whole Bronze table, we need to know: "What was the last event I processed?"

We use a High Watermark (the maximum ingestion_timestamp currently in Silver).

The logic flow is the following:

1. Find MAX(ingested_at) in Silver
2. Select all rows in Bronze where ingestion_timestamp > Watermark
3. Merge them into Silver

The first Flyte workflow in workflows.py implements this logic. You can run it with

```
pyflyte run workflows.py shopnow_incremental_sync
```

You can run the workflow multiple times and check the execution time. In subsequent times (for the same kafka events) it should be much faster, although the difference might not be noticeable if we are using small data...

The Gold Layer (Batch Aggregates)

The Gold layer represents data products ready for consumption by BI tools or ML models. For ShopNow, we want a Daily Sales Summary that calculates total revenue and number of unique customers per day. In the Gold layer, we aggregate data into a higher "grain" (one row per day per item). Let's start by creating the table in Trino:

```
CREATE TABLE iceberg.gold.daily_sales_summary (  
  sale_date DATE,  
  item_id VARCHAR,  
  total_revenue DOUBLE,  
  order_count BIGINT,  
  unique_customers BIGINT,  
  updated_at TIMESTAMP(6) WITH TIME ZONE  
)  
WITH (  
  format = 'PARQUET',  
  partitioning = ARRAY['year(sale_date)']  
);
```

Since the Gold layer is often a summary of the Silver layer, we can use a "Full Refresh" or a "Windowed Batch" logic. For this tutorial, we will implement a batch task that re-computes the last 7 days to account for any late data that was promoted to Silver. This is done in the second workflow. You can run it with

```
pyflyte run streaming_pipeline.py shopnow_batch_sync
```